


```

--: POSITIVE:= 1;-----| ETAT D ANALYSEUR INITIALISE A 1 AP:= INTEGER; ACTION:= INTEGER; ASYM:= AC_BYTE; NBR_OF_SYLS:= NATUR
AL;-----| NUMBER OF SYLLABLES TO BE-POPPED ZERO_BYTE:= constant AC_BYTE:= 0;-----| procedure DEBUG_PRINT( TXT : STRING )
-----| is begin for I in 2 .. SP loop PUT( " " ); end loop; PUT( 's' & POSITIVE'IMAGE( STATE ) & 's' ); if 2*S
P + TXT'LENGTH > 77 then PUT_LINE( TXT( TXT'FIRST .. 77 - 2 * SP ) ); else begin PUT_LINE( TXT ); end if; end DEBUG_PRINT;-----
-----| procedure DEBUG_PRINT( V : TREE ) is begin DEBUG_PRINT( PRINT_NAME( V ) ); end DEBUG_PRINT;-----| b
egin LTYPE := LT_END_MARK;-----| INITIALISER A LIGNE VIDE SOURCEPOS := TREE_VOID;-----| INITIALISER LA POSITION SOURCE A VID
E GET_TOKEN;-----| ALLER-CHERCHER UN LEXEME STACK( 1 ).STATE := 1; STACK( 1 ).SRCPOS := SOURCEPOS; STACK( 2 ).SRCPOS := SOUR
CEPOS; SSITOP := 0;-----| START WITH EMPTY-SEMANTIC STACK loop AP := GRMR_TBL.GRMR.AC_SYM( AP ); if AP <= 0 then
0130 ACTION := AP; else -----| POINTS TO-SHIFT STUFF loop ASYM := GRMR_TBL.GRMR.AC_SYM( AP ); exit when ASYM = ZERO_BYTE or else
ASYM = LEX_TYPE'POS( TOKENSYM ); AP := AP + 1; end loop; ACTION := INTEGER( GRMR_TBL.GRMR.AC_TBL( AP ) ); if AC
TION > 0 then-----| CAN'T BE SEMANTICS SINCE DIDN'T INDIRECT if DEBUG_PARSE then if LTYPE in LT_WITH_SEMANTICS or else LTYPE = L
T_ERROR then DEBUG_PRINT( LEX_IMAGE( LTYPE ) & " " & TOKEN_STRING ); else DEBUG_PRINT( TOKEN_STRING ); end if; if AC
-SEMANTIC STACK IF THIS TOKEN HAS SEMANTICS if LTYPE in LT_WITH_SEMANTICS then if LTYPE = LT_NUMERIC_LIT then AUA := ( TOKEN_ELMT, SOU
RCEPOS, STORE_TEXT( TOKEN_STRING ) ); else AUA := ( TOKEN_ELMT, SOURCEPOS, STORE_SYM( TOKEN_STRING ) ); end if; if
if LTYPE = LT_END_MARK then-----| ARRIVE EN FIN DE COMPILATION if SP /= 2 then PUT_LINE( "FIN COMPILE MAIS SP = " & INTEGER'IM
AGE( SP ) ); end if; if SSITOP /= 1 then PUT( "FIN COMPILE MAIS SSITOP = " & INTEGER'IMAGE( SSITOP ) ); else AUA := SEMSTAK( 1 ); if A
UA.KIND /= NODE_ELMT then PUT_LINE( "FIN COMPILE MAIS SEMSTAK(1) PAS UN NOEUD. " ); else-----| SAUVER L'ARBRE SYNTAXIQUE-DANS LE XD_ST
RUCTURE DU USER_ROOT D( XD_STRUCTURE, USER_ROOT, AUA.ELMT ); if DEBUG_PARSE then PRINT_NODE( D( XD_STRUCTURE, USER_ROOT ) ); end if; e
0140 nd if; end if; exit; end if; SP := SP + 1; STATE := ACTION; STACK( SP ).STATE := ACTION; STACK( SP ).SRC
POS := SOURCEPOS; STACK( SP+1 ).SRCPOS := SOURCEPOS; GET_TOKEN; elsif ACTION = 0 then-----| ERREUR DE SYNTAXE
ERROR( SOURCEPOS, "ERREUR DE SYNTAXE - " & SLINE.BDY( F_COL..E_COL ) );-----| INSERE UN NOEUD-ERREUR ET-AFFICHE UN MESSAGE exit; el
se-----| SEMANTIC AND REDUCE ACTIONS NODE_CREATED := FALSE; loop if ACTION > -10000 then -----| TRANSFER TO SEMANTIC ACTION TABLE
AP := --ACTION; -- TRANSFER-IN TABLE loop ACTION := INTEGER( GRMR_TBL.GRMR.AC_TBL( AP ) ); exit when ACTION <= 0; BUILD_TREE( ACT
ION, AP );-----| CONSTRUCTION DE-L'ARBRE INCREMENTE AP EN INTERNE end loop; end if; if ACTION > -30000 and ACTION <= -10000 then
--REDUCE ACTION := --ACTION - 10000; NBR_OF_SYLS := ACTION/1000; ACTION := ACTION mod 1000; -- I.E., RULE SP := SP - NBR_OF_SYL
S; -- POP THE STACK STATE := STACK( SP ).STATE; SEMSTAK( SSITOP ).SPOS := STACK( SP+1 ).SRCPOS; if NODE_CREATED and then SEMSTAK( SSITOP ).K
IND = NODE_ELMT and then SEMSTAK( SSITOP ).ELMT /= TREE_VOID then D( LX_SRCPOS, SEMSTAK( SSITOP ).ELMT, SEMSTAK( SSITOP ).SPOS ); end
d if; -- FIND GOTO FOR NONTERMINAL IN-THIS STATE AP := GRMR_TBL.GRMR.AC_TBL( STATE ); loop AP := AP + 1; ASYM := GRMR_TBL.GRMR
0150 R.AC_SYM( AP ); if ASYM = ZERO_BYTE then PUT_LINE( "!! ***** NONTER GOTO NOT FOUND. " ); raise PROGRAM_ERROR; end if;
exit when INTEGER( ASYM ) = ACTION; end loop; STATE := INTEGER( GRMR_TBL.GRMR.AC_TBL( AP ) ); SP := SP + 1; STACK( SP ).STATE := STATE;
if NBR_OF_SYLS = 0 then-----| NULLABLE REDUCTION; SRCPOS-NOT ALREADY THERE STACK( SP ).SRCPOS := SOURCEPOS; end if; STACK( SP+1 ).SRCPOS
:= SOURCEPOS; exit; else PUT_LINE( "!! PARSE_TABLE_ERROR " ); raise PROGRAM_ERROR; end if; end loop; end if; end loop;
end PARSE_COMPILATION;-----| begin READ_PARSE_TABLES;-----| TABLES DE LA GRAMMAIRE LUES DANS PARSE.BIN OPEN( IFILE, IN_FIL
E, PATH_TEXTE & NOM_TEXTE );-----| OUVRIER LE FICHIER SOURCE A COMPILER PUT( "ada83 compiling " & PATH_TEXTE & NOM_TEXTE ); CREATE_IDL_TREE_FI
LE( IDL.LIB_PATH( 1..LIB_PATH_LENGTH ) & "$$.TMP " );-----| CREER-LE FICHIER D'ARBRE (AVEC SON NOEUD RACINE DE TYPE-DN_ROOT) USER_ROOT := MAKE( DN
_USER_ROOT );-----| CREER-UN NOEUD RACINE SECONDAIRE DU-TYPE DN_USER_ROOT D( XD_USER_ROOT, TREE_ROOT, USER_ROOT );-----| NOEUD-USER_ROOT-
DANS LE CHAMP XD_USER_ROOT DU-NOEUD TREE_ROOT D( XD_SOURCENAME, USER_ROOT, STORE_TEXT( NOM_TEXTE ) );-----| NOM DU SOURCE DANS LE CHAMP XD_XOURC
ENAME DU NOEUD USER_ROOT PARSE_COMPILATION;-----| EFFECTUER LA PHASE D'ANALYSE SYNTAXIQUE DU SOURCE D( XD_SOURCE_LIST, TREE_ROOT, SOURCE
LIST.FIRST );-----| STOCKE LA LISTE-SOURCE_LIST DANS L'ATTRIBUT XD_SOURCE_LIST CLOSE( IFILE ); CLOSE_PAGE_MANAGER;-----| FERMER LE FICH
IER ARBRE exception when NAME_ERROR => PUT_LINE( "FILE NOT FOUND : " & PATH_TEXTE & NOM_TEXTE ); raise; end PAR_PHASE;-----|
3-4-5-6-7-8-9-10-11-12

```